

Doc. no:	P0781R0
Date:	2017-09-25
Reply to:	Erich Keane

A Modern C++ Signature for main

Introduction

One of the greatest accomplishments of the ISO C++ Committee over the past decade was to provide easy to use and powerful zero-cost abstractions to painful C-isms in the language. A very useful benefit of these abstractions is that they make common operations easy and consistent.

However, one of the last vestiges of C left in C++ is also both the most difficult to educate new programmers, and an incredibly error prone one to knowledgeable programmers. This of course, is the available signatures of the application entry function, aka main. The meaningful signature of main dates back to the earliest versions of C. This paper proposes adding an additional signature for the main function, starting with some guidelines that should be used to select a replacement, as well as a few potential options.

Justification

First, let's consider a somewhat common usage of the useful signature of main:

```
int main(int argc, char** argv){
    for (size_t i = 0; i < argc; ++i) {
        char *Arg = argv[i];
        size_t ArgSize = strlen(Arg);
        // some usage of this character array...
    }
}
```

One thing that you may take from this pessimized and contrived example is the horrible amount of C-isms and otherwise terrible set of functions that the programmer is immediately being exposed to. For a student of Modern C++, one can imagine how terrifying this is for an otherwise simple operation. This requires familiarity of C-Arrays, pointer decay, C-string functions, and even traditional for-loops!

Any experienced C++ programmer would likely be disgusted by the same issues, and would immediately wrap this in some other type, such as `boost::program_options`. Even so, this is a sizable dependency for a smaller application that is perhaps significantly more complex than required.

Goals for a Solution

The author of this paper has two main goals for this proposal:

1. Make the program command line arguments easy to use for experienced programmers in a way that embraces Modern C++ features.
2. Make the program command line arguments consistent enough with the remainder of Modern C++ that students to the language will have an immediate understanding of how to accept them.

Immediately, removing all mention of traditional-arrays, pointers, C-string handling, and traditional for-loops are a useful goal. This paper proposes an alternative syntax that would look more like the following example:

```
int main(const some_container<const some_string_type> args){
    for (auto Arg : args) {
        // some usage of this character array...
    }
}
```

The primary benefit of this version is that a container of string-types is likely the most familiar type of structure to both experts and beginners alike, providing a simple to use, self-explanatory structure for programmers of all levels. Gone are all of the c-isms, which have all been replaced with memory safe alternatives that are significantly more terse and self-explanatory.

In addition to the above form, this paper proposes a number of options for `some_container` and `some_string_type`. The goals of these are listed below:

`some_container`

1. Well ordered: The container should maintain the actual order of the parameters listed on the command line.
2. Iterable: Range-for and iterators are incredibly useful and consistent in Modern C++, so looping through the parameters is necessary.
3. Contiguous Storage: This minimizes storage, and provides the best chance for operating system support as a zero-cost-abstraction.

`some_string_type`

1. Constructible from a `char*`: Currently the majority of OSs provides an array of character pointers, so something that could be easily be constructed from one of those is likely to provide the best performance.
2. Convertible to a standard C++ string type: A vast majority of operations and algorithms in current C++ code accept `std::string`. A type that either is, or is easily convertible to one of these is optimal.

Suggested Solution

The author of this paper suggests `std::initializer_list<std::string_view>` for a number of reasons, in addition to the thoughts above. Firstly, `std::initializer_list` is already recognized and specially constructed by the compiler. It is a lightweight type that can be trivially mapped to an existing section of memory, so this provides the most flexibility for the implementation details. Additionally, `std::string_view` is ALSO incredibly lightweight, and can be mapped easily from an existing section of memory. It also has the advantage of being trivially copyable, simply convertible to `std::string`.

This solution, however, is not quite perfect.

First, on most operating systems this signature requires an allocation of size "argc", since the current `char**` signature cannot be trivially copied to a `std::initializer_list` type. However, the author would like to point out that the OS ALREADY knows the length, so the OS is not required to provide additional calculation in order to provide the process with a list of Pointer/Integer pairs rather than just a list of Pointers.

Secondly, this signature currently requires an increased startup cost, since the length of each parameter must be calculated. However, the programmer is likely to require this calculation anyway. There is the slight risk that an individual executing a function using this signature could send a very large amount of command-line parameter characters. The author of this paper believes that this is an acceptable risk (since at worst, it doubles the launch-cost in this situation). If the programmer believes this is a viable risk, they are still welcome to use one of the previous signatures.

Finally, `initializer_list` has two minor issues. First, it does not have a random access iterator. This case is believed to be fairly minor, since the ordering of parameters is typically more important than ordinal location. Again, if the alternate is completely mandatory, the existing signature is still available. Secondly, the `initializer_list` does not have a constructor that would work in this situation. However, it is already a type that is magically created by the constructor, so one more condition where this happens seems acceptable.

Past/Potential Critisms

What about `array_view`?

This type actually has a number of advantages of `initializer_list` plus would add the random access indexing, however it currently does not exist in the standard. Not proposed here, but perhaps possible for a future paper would be to add random access to `initializer_list`

What about `std::string/std::vector`?

These types have two big costs that are likely not acceptable. First, they require an allocator, which could potentially require state which would require a guaranteed execution order compared to global initialization. Secondly, they would likely prevent an intelligent operating system designer from changing the entry-function format of the OS to better match the language entry function.

What happens if the user didn't include `initializer_list/string_view` headers/modules?

The author believes that this should be an error condition. However, others have argued that the compiler should materialize these includes/imports if necessary. The author has no issue with this behavior.

Why is it a `const string_view`?

As this functionality is meant for typical usage and to prevent common errors, this paper proposes disallowing modification of the command line arguments by default. The existing form of the entry function can be used by those wishing to modify their arguments.

What about `argv[argc]`?

The standard (both C and C++) state that this should contain 0. This paper proposes to make this value not part of the `initailizer_list`, as it is not terribly useful for programmers, and confusing at best for beginner programmers.